Introduction to Deep Learning

Session 4: Computation Graphs & Backpropagation

April 2026

Magda Gregorová - magda.gregorova@thws.de

Technical University
of Applied Sciences
Würzburg-Schweinfurt

Licensed according to CC-BY-SA 4.0

Lecture Overview

1. Computation Graphs

2. Backpropagation

Forward and Backward Pass

Forward pass - evaluate network $\rightarrow$ get predictions $\hat{y}$ $\rightarrow$ compute loss $\mathcal{L}$

$$\mathbf{x} = \underbrace{\mathbf{h}^{(0)} \xrightarrow{\boldsymbol{\Theta}^{(1)}} \mathbf{h}^{(1)} \xrightarrow{\boldsymbol{\Theta}^{(2)}} \cdots \xrightarrow{\boldsymbol{\Theta}^{(L)}} \mathbf{h}^{(L)}}_{f_\theta} = \hat{y} \xrightarrow{\ell} \mathcal{L}$$

Backward pass - propagate gradients from loss back to parameters: **Backpropagation**

$$\nabla_\theta \mathcal{L}(\boldsymbol{\theta}) = \underbrace{\frac{\partial \mathcal{L}}{\partial \boldsymbol{\theta}} \xleftarrow{\boldsymbol{\Theta}^{(1)}} \frac{\partial \mathcal{L}}{\partial \mathbf{h}^{(1)}} \xleftarrow{\boldsymbol{\Theta}^{(2)}} \cdots \xleftarrow{\boldsymbol{\Theta}^{(L)}} \frac{\partial \mathcal{L}}{\partial \mathbf{h}^{(L)}}}_{\nabla_\theta} \xleftarrow{\ell} \mathcal{L}$$

Gradient: $\nabla_\theta \mathcal{L}(\boldsymbol{\theta}) = \dfrac{\partial \mathcal{L}}{\partial \boldsymbol{\theta}}$ - one partial derivative per parameter

Computation Graphs - Motivation

Idea: decompose complex functions into simple atomic operations

Why?

- Gradients of complex functions are hard - gradients of atomic operations are easy
- Intermediate values from forward pass can be reused in backward pass

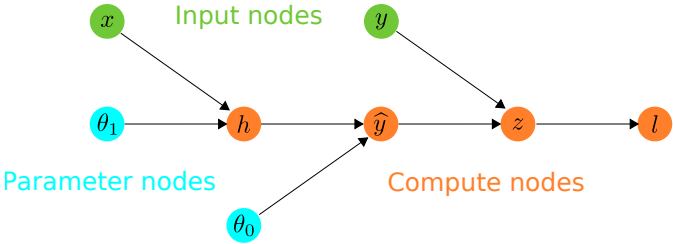
Graph structure:

- Input nodes - fixed data (x, y)
- Parameter nodes - learnable parameters (θ)
- Compute nodes - elementary operations, each with a simple local derivative

Computation Graphs - Example

Example: linear regression with MSE loss: $\ell(x, y, \boldsymbol{\theta}) = (\theta_0 + \theta_1 x - y)^2$

Each step - simple atomic operation with easy derivative.



Decomposition:

(1) $h = \theta_1 x$

(2) $\hat{y} = \theta_0 + h$

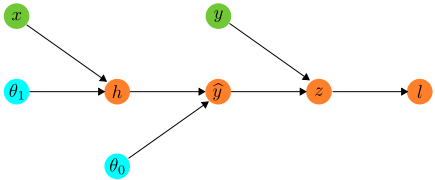
(3) $z = \hat{y} - y$

(4) $\ell = z^2$

Computation Graphs - Flexible Level of Detail

Same function, different decompositions: $\ell(x, y, \boldsymbol{\theta}) = (\theta_0 + \theta_1 x - y)^2$

Fine-grained



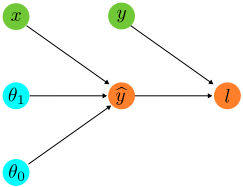
(1) $h = \theta_1 x$

(2) $\hat{y} = \theta_0 + h$

(3) $z = \hat{y} - y$

(4) $\ell = z^2$

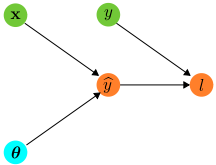
Coarser



(1) $\hat{y} = \theta_0 + \theta_1 x$

(2) $\ell = (\hat{y} - y)^2$

Vectorized



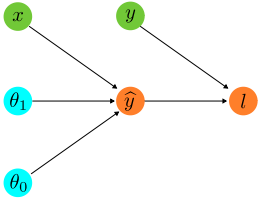
(1) $\hat{y} = \boldsymbol{\theta}^\top \mathbf{x}$

(2) $\ell = (\hat{y} - y)^2$

Computation Graphs - Flexible Operations

Same graph structure, different models:

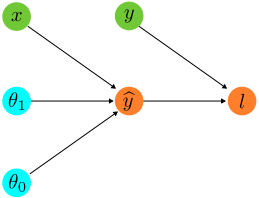
Linear regression + MSE



(1) $\hat{y} = \theta_0 + \theta_1 x$

(2) $\ell = (\hat{y} - y)^2$

Logistic regression + BCE

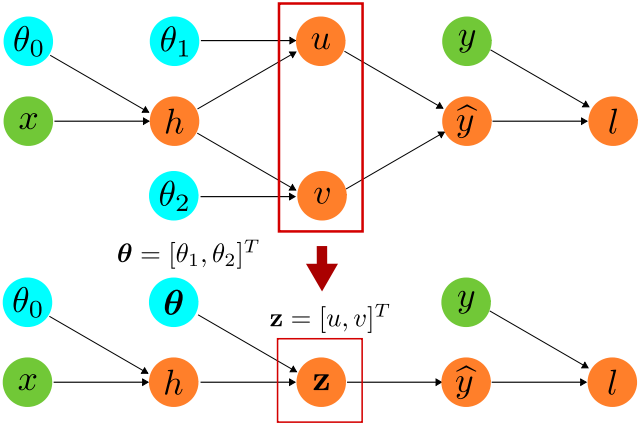


(1) $\hat{y} = \sigma(\theta_0 + \theta_1 x)$

(2) $\ell = -y \log \hat{y} - (1 - y) \log(1 - \hat{y})$

Computation Graphs - Scalar vs Vector

Scalar notation (one neuron at a time) vs **vector notation** (whole layer at once):



Scalar

(1) $h = h(x, \theta_0)$

(2) $u = u(h, \theta_1)$

$v = v(h, \theta_2)$

(3) $\hat{y} = \hat{y}(u, v)$

(4) $\ell = \ell(\hat{y}, y)$

Vector

(1) $h = h(x, \theta_0)$

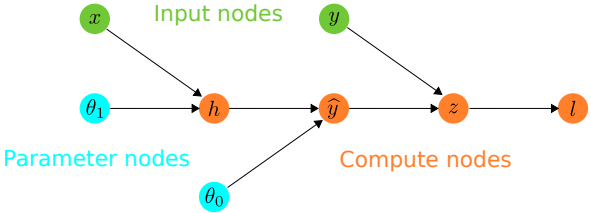
(2) $\mathbf{z} = \mathbf{z}(h, \boldsymbol{\theta})$

(3) $\hat{y} = \hat{y}(\mathbf{z})$

(4) $\ell = \ell(\hat{y}, y)$

Forward and Backward Pass (Backpropagation)

Forward pass - prediction and loss; **Backward pass** - parameter gradients



Forward pass

$$\mathbf{x} \longrightarrow \mathbf{h}^{(1)} \longrightarrow \cdots \longrightarrow \mathbf{h}^{(L)} \longrightarrow \ell$$

Backpropagation

$$\frac{\partial \ell}{\partial \boldsymbol{\theta}} \longleftarrow \cdots \longleftarrow \frac{\partial \ell}{\partial \mathbf{h}^{(L)}} \longleftarrow \ell$$

Two Key Rules

Goal: compute $\nabla_{\boldsymbol{\theta}} \mathcal{L}(\boldsymbol{\theta})$ for the full dataset

1) Sum rule - gradient of average = average of gradients:

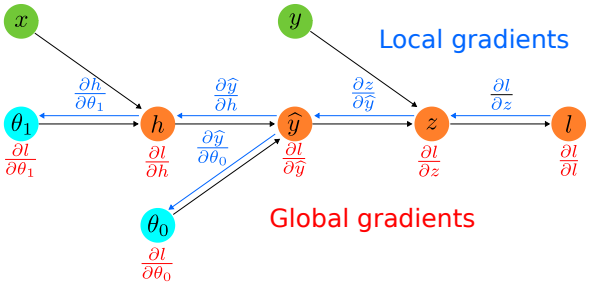
$$\nabla_{\boldsymbol{\theta}} \mathcal{L}(\boldsymbol{\theta}) = \nabla_{\boldsymbol{\theta}} \frac{1}{N} \sum_{i=1}^{N} \ell(\mathbf{x}^{(i)}, y^{(i)}, \boldsymbol{\theta}) = \frac{1}{N} \sum_{i=1}^{N} \nabla_{\boldsymbol{\theta}} \ell(\mathbf{x}^{(i)}, y^{(i)}, \boldsymbol{\theta})$$

$\Rightarrow$ compute gradient for a single sample, then average

2) Chain rule - THE rule for backprop:

$$\frac{d}{dx} f(g(x)) = \frac{df}{dg} \frac{dg}{dx} \qquad \frac{d}{dx} f(g_1(x), \ldots, g_m(x)) = \sum_{i=1}^{m} \frac{\partial f}{\partial g_i} \frac{dg_i}{dx}$$

Local vs Global Gradients



Local gradient (easy — per node):

$$\frac{\partial \text{out}}{\partial \text{in}}$$

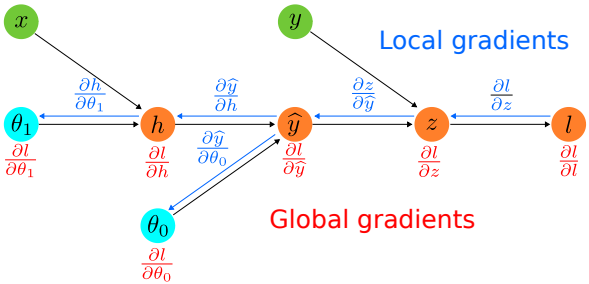
Can compute during forward pass

Global gradient (chain rule):

$$\frac{\partial \ell}{\partial \text{in}} = \frac{\partial \ell}{\partial \text{out}} \cdot \frac{\partial \text{out}}{\partial \text{in}}$$

Goes backward: incoming global × local = new global for the left

Backprop Example: Linear Regression (Symbolic)



Forward pass ↓

(1) $h = \theta_1 x$

(2) $\hat{y} = \theta_0 + h$

(3) $z = \hat{y} - y$

(4) $\ell = z^2$

Local gradients ↓

(1) $\partial h / \partial \theta_1 = x$

(2) $\partial \hat{y} / \partial \theta_0 = 1$

$\partial \hat{y} / \partial h = 1$

(3) $\partial z / \partial \hat{y} = 1$

(4) $\partial \ell / \partial z = 2z$

Global gradients ↑

(4) $\partial \ell / \partial \theta_1 = 2z \cdot x = 2zx$

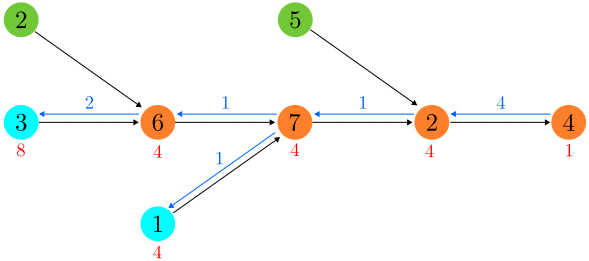
$\partial \ell / \partial \theta_0 = 2z \cdot 1 = 2z$

(3) $\partial \ell / \partial h = 2z \cdot 1 = 2z$

(2) $\partial \ell / \partial \hat{y} = 2z \cdot 1 = 2z$

(1) $\partial \ell / \partial z = 1 \cdot 2z = 2z$

Backprop Example: Linear Regression (Numerical)



Concrete values: $x = 2,\ y = 5,\ \theta_0 = 1,\ \theta_1 = 3$

Forward pass ↓

(1) $h = \theta_1 x = 3 \cdot 2 = 6$

(2) $\hat{y} = \theta_0 + h = 1 + 6 = 7$

(3) $z = \hat{y} - y = 7 - 5 = 2$

(4) $\ell = z^2 = 2^2 = 4$

Local gradients ↓

(1) $\partial h / \partial \theta_1 = x = 2$

(2) $\partial \hat{y} / \partial \theta_0 = 1$

$\partial \hat{y} / \partial h = 1$

(3) $\partial z / \partial \hat{y} = 1$

(4) $\partial \ell / \partial z = 2z = 4$

Global gradients ↑

(4) $\partial \ell / \partial \theta_1 = \partial \ell / \partial h \cdot x = 4 \cdot 2 = 8$

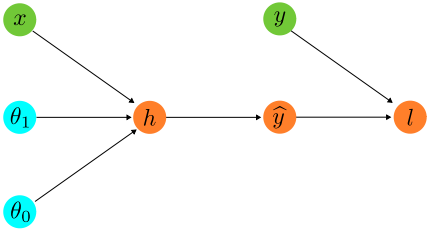
$\partial \ell / \partial \theta_0 = \partial \ell / \partial h \cdot 1 = 4 \cdot 1 = 4$

(3) $\partial \ell / \partial h = \partial \ell / \partial \hat{y} \cdot 1 = 4 \cdot 1 = 4$

(2) $\partial \ell / \partial \hat{y} = \partial \ell / \partial z \cdot 1 = 4 \cdot 1 = 4$

(1) $\partial \ell / \partial z = 1 \cdot 2z = 1 \cdot 4 = 4$

Backprop Example: Logistic Regression



Forward pass ↓

(1) $h = \theta_0 + \theta_1 x$

(2) $\hat{y} = \sigma(h)$

(3) $\ell = -y \log \hat{y} - (1-y)\log(1-\hat{y})$

Local gradients ↓

(1) $\partial h / \partial \theta_0 = 1$

$\partial h / \partial \theta_1 = x$

(2) $\partial \hat{y} / \partial h = \sigma(h)(1-\sigma(h))$

(3) $\partial \ell / \partial \hat{y} = \dfrac{\hat{y} - y}{\hat{y}(1-\hat{y})}$

Global gradients ↑

(3) $\partial \ell / \partial \theta_0 = \partial \ell / \partial h \cdot 1$

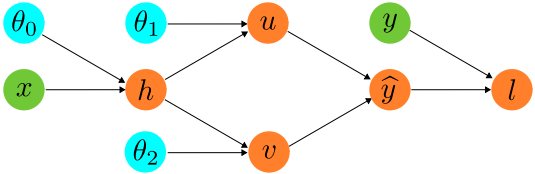
$\partial \ell / \partial \theta_1 = \partial \ell / \partial h \cdot x$

(2) $\partial \ell / \partial h = \partial \ell / \partial \hat{y} \cdot \partial \hat{y} / \partial h$

(1) $\partial \ell / \partial \hat{y} = 1 \cdot \dfrac{\hat{y} - y}{\hat{y}(1-\hat{y})}$

Key insight: same graph structure, only the node functions change

Backprop: Multiple Neurons in a Layer



Forward pass ↓

(1) $h = h(x, \theta_0)$

(2) $u = u(h, \theta_1)$

$v = v(h, \theta_2)$

(3) $\hat{y} = \hat{y}(u, v)$

(4) $\ell = \ell(\hat{y}, y)$

Local gradients ↓

(1) $\partial h / \partial \theta_0$

(2) $\partial u / \partial \theta_1, \quad \partial u / \partial h$

$\partial v / \partial \theta_2, \quad \partial v / \partial h$

(3) $\partial \hat{y} / \partial u, \quad \partial \hat{y} / \partial v$

(4) $\partial \ell / \partial \hat{y}$

Global gradients ↑

(4) $\partial \ell / \partial \theta_0 = \partial \ell / \partial h \cdot \partial h / \partial \theta_0$

(3) $\partial \ell / \partial h = \partial \ell / \partial u \cdot \partial u / \partial h + \partial \ell / \partial v \cdot \partial v / \partial h$

$\partial \ell / \partial \theta_1 = \partial \ell / \partial u \cdot \partial u / \partial \theta_1$

$\partial \ell / \partial \theta_2 = \partial \ell / \partial v \cdot \partial v / \partial \theta_2$

(2) $\partial \ell / \partial u = \partial \ell / \partial \hat{y} \cdot \partial \hat{y} / \partial u$

$\partial \ell / \partial v = \partial \ell / \partial \hat{y} \cdot \partial \hat{y} / \partial v$

(1) $\partial \ell / \partial \hat{y} = 1 \cdot \partial \ell / \partial \hat{y}$

When a node feeds multiple downstream nodes, gradients **accumulate** (sum).

Technical Implementation

Each node stores:

- `.val` - forward value
- `.grad` - accumulated gradient

Each compute node implements:

- Forward function
- Local gradient function

Forward pass

```
x.val, y.val = data
t0.val, t1.val = init
h.val = t1.val * x.val
yp.val = t0.val + h.val
z.val = yp.val - y.val

l.val = z.val ** 2
```

Backward pass

```
l.grad = 1
z.grad = l.grad * 2*z.val
yp.grad = z.grad * 1
t0.grad = yp.grad * 1
h.grad = yp.grad * 1

t1.grad = h.grad * x.val
```

From For Loops to Vectorization

Elementwise for loops

```
# Forward
for i:  z.val[i] = b.val[i]
for i,j:  z.val[j] += W.val[i,j] * x.val[i]
for i:  a.val[i] = act(z.val[i])

# Backward
for i:  z.grad[i] = a.grad[i] * dact(z.val[i])
for i,j:  W.grad[i,j] = z.grad[j] * x.val[i]

for i:  b.grad[i] = z.grad[i]
```

Vectorized

```
# Forward
z.val = W.val @ x.val + b.val
a.val = act(z.val)

# Backward
z.grad = a.grad * dact(z.val)
W.grad = z.grad[:,None] * x.val[None,:]

b.grad = z.grad
```

Same computation, ~100× faster on GPU

Summary

Computation graphs

- Decompose complex functions into atomic operations
- Forward pass - function evaluation
- Backward pass - gradient computation
- Flexible level of detail and definition of operations

Backpropagation

- Computes gradients - does **not** optimize
- Two rules: sum rule + chain rule
- Local gradients easy per node
- Global gradients via chain rule

Implementation

- Each node stores `.val` and `.grad`
- Compute nodes only need to be local-aware:
  - inputs and outputs
  - forward function
  - local gradient function
- Vectorize for efficiency (~100× faster on GPU)

Next

- PyTorch autograd does all of this automatically
- `loss.backward()` = one backward pass